

AASTool Backend Architecture

AASTool — Backend Architecture & Services Documentation

Enterprise Edition | Version 1.0.0

Table of Contents

- 1. [Technology Stack](#)
 - 2. [Architecture Overview](#)
 - 3. [Entry Point: index.ts](#)
 - 4. [Data Source Configuration](#)
 - 5. [Entity Model](#)
 - 6. [Controller Layer](#)
 - 7. [Service Layer](#)
 - 8. [Calculation Engine](#)
 - 9. [Migrations](#)
 - 10. [Database Seeding](#)
 - 11. [Tests](#)
 - 12. [Docker Configuration](#)
-

Technology Stack

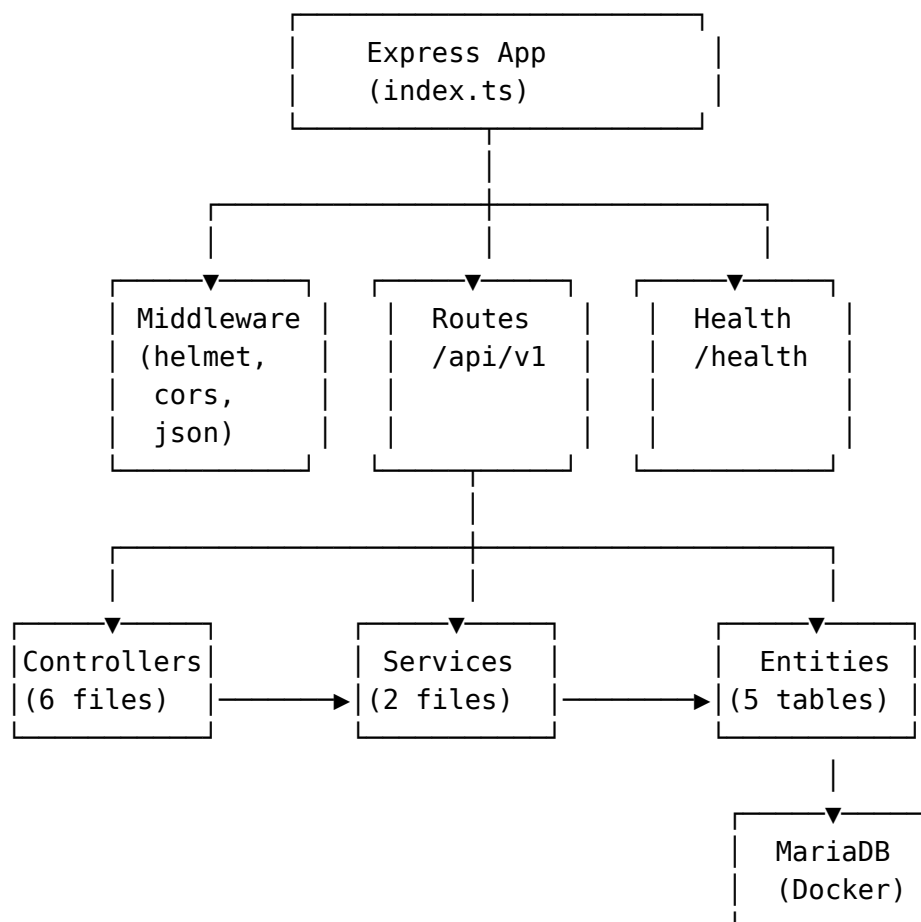
Technology	Version	Purpose
Node.js	22	Runtime
TypeScript	5.x	Type safety
Express	4.18	HTTP framework
TypeORM	0.3.17	ORM (Object-Relational Mapper)
MariaDB	10.11	Relational database
ts-node	10.9	TypeScript execution

Technology	Version	Purpose
Jest	Latest	Testing framework
Docker Compose	3.8	Container orchestration

Module System

The backend uses **CommonJS** ("module": "commonjs"). Imports use `require()`-style syntax compiled by TypeScript.

Architecture Overview



Request Lifecycle

1. HTTP Request arrives at Express
2. Global middleware: helmet (security headers) → cors → express.json()
3. JSON parse error middleware: validates JSON body
4. Route handler (controller function):
 - a. Validate request body/params
 - b. Call service layer for business logic

- c. Query database via TypeORM Repository
 - d. Return JSON response { ok: true/false, data/error }
-

Entry Point: index.ts

Server Startup Sequence

1. Create Express app
2. Apply middleware (helmet, cors, json, error handler)
3. Register all routes
4. Initialize DataSource (connect to MariaDB)
5. Run metadata validation
6. Start listening on PORT (env, default 4000)
7. Health endpoints report APP_VERSION (env, default '1.0.0')

Routes Registered

Method	Path	Handler	S
GET	/health	Inline	index.t
GET	/api/v1/health	Inline	index.t
POST	/api/v1/evaluate	evaluateController	evaluati
GET	/api/v1/criteria	getCriteria	dataCon
POST	/api/v1/criteria	createCriterion	dataCon
GET	/api/v1/criteria/:code	getCriterionByCode	dataCon
GET	/api/v1/building-types	getBuildingTypes	dataCon
GET	/api/v1/neb-thresholds	getNebThresholds	dataCon
GET	/api/v1/config	getConfig	dataCon
GET	/api/v1/disability-types	getDisabilityTypes	dataCon
GET	/api/v1/assessment-dimensions	getAssessmentDimensions	dataCon
GET	/api/v1/buildings	getBuildings	buildin
GET	/api/v1/buildings/:id	getBuilding	buildin
POST	/api/v1/assessments	createAssessment	assessm
GET	/api/v1/assessments/:id	getAssessment	assessm
GET	/api/v1/reports	getReports	reports
POST	/api/v1/login	login	authCon

Health Endpoint

GET /health and GET /api/v1/health return:

```
{
  "status": "ok",
  "uptime": 3600,
  "database": "connected",
  "dbLatencyMs": 2,
  "version": "1.0.0",
  "timestamp": "2026-06-15T12:00:00.000Z"
}
```

On database failure:

```
{
  "status": "degraded",
  "uptime": 3600,
  "database": "disconnected",
  "dbLatencyMs": null,
  "version": "1.0.0",
  "timestamp": "2026-06-15T12:00:00.000Z"
}
```

Middleware Stack

```
// Security headers
app.use(helmet());

// Cross-origin requests
app.use(cors());

// JSON body parsing
app.use(express.json());

// JSON parse error handler
app.use((err, _req, res, next) => {
  if (err?.type === 'entity.parse.failed')
    return res.status(400).json({ ok: false, error:
'Invalid JSON body' });
  if (err instanceof SyntaxError && 'body' in err)
    return res.status(400).json({ ok: false, error:
'Malformed JSON' });
  return next(err);
});
```

Data Source Configuration

data-source.ts

```
const AppDataSource = new DataSource({
  type: 'mariadb',
```

```
    url: process.env.DATABASE_URL,
    entities: [Criterion, BuildingType, Evaluation,
NebThreshold, Config],
    migrations: [...],
    synchronize: false,    // NEVER auto-sync in production
    logging: false,
  });
```

Critical: DATABASE_URL is required. The DataSource throws an Error if not set:

```
if (!process.env.DATABASE_URL) {
  throw new Error('Environment variable DATABASE_URL is
required');
}
```

Connection String Format

mysql://user:password@host:3306/database

Environment Variables

All hardcoded values have been externalized to environment variables:

Variable	Default	Used In	Purpose
DATABASE_URL	(required)	data-source.ts	MariaDB connect string
PORT	4000	index.ts	Server listen port
APP_VERSION	1.0.0	index.ts	Version reported /health
JWT_SECRET	(required)	authController.ts	HMAC-SHA256 signing
JWT_EXPIRES_IN	3600	authController.ts	Token lifetime (second)
LOGIN_REDIRECT	/dashboard	authController.ts	Post-log redirect path
ASSESSMENT_STATUS	Completed	buildingsController.ts	Default assessn status

Variable	Default	Used In	Purpose
REPORT_LOCALE	en-US	reportsController.ts	Date format locale
REPORT_TITLE_PREFIX	Accessibility Report	reportsController.ts	Report title prefix

Entity Model

1. Criterion (criteria table)

Represents a single evaluation criterion from the EC framework.

Column	Type	Description
id	UUID (PK)	Auto-generated primary key
code	VARCHAR (unique)	Criterion code, e.g. "EC1.1.3"
name	VARCHAR	Short name
definition	TEXT	Full definition of the criterion
justification	TEXT	Why this criterion matters
value	FLOAT	Base weight value (Wij)
disability	INT	Disability type ID (1-5)
dimension	INT	Assessment dimension ID (1-5)
score	INT (default: 3)	Default score
levels	JSON	Array of 5 level descriptions (Level 1-5)

Code Format: EC{disability}.{dimension}.{sequence} — e.g., EC1.1.3 = Physical DT, Wayfinding AD, 3rd criterion.

2. BuildingType (building_types table)

Represents a building category with its weighting configuration.

Column	Type	Description
id	UUID (PK)	Auto-generated primary key
name	VARCHAR (unique)	Building type name (e.g., "Commercial Buildings")
disability_weights	JSON	Array of 5 weights (DT1-DT5)
dimension_weights	JSON	Array of 5 weights (AD1-AD5)

3. Evaluation (evaluations table)

Stores the results of an accessibility assessment.

Column	Type	Description
id	UUID (PK)	Auto-generated primary key
user_id	VARCHAR (nullable)	Optional user identifier
building_type	VARCHAR	Building type name
scores	JSON	All criterion scores { code: score }
result	JSON	Full evaluation result object
obs	DOUBLE (nullable)	Overall Building Score (0-100)
neb_class	VARCHAR (nullable)	NEB classification (A+, A, B, C, D)
average_raw_score	DOUBLE (nullable)	Average score across all criteria
created_at	DATETIME	Auto-generated timestamp

4. NebThreshold (neb_thresholds table)

Maps OBS ranges to NEB classifications.

Column	Type	Description
id	UUID (PK)	Auto-generated primary key
min	INT	Minimum OBS for this band
neb_class	VARCHAR	Class label (A+, A, B, C, D)
equivalent	VARCHAR	Human-readable equivalent
meaning	TEXT	What this class means
neb_score	INT	Numeric score (5 down to 1)
ordinal	INT	Sort order

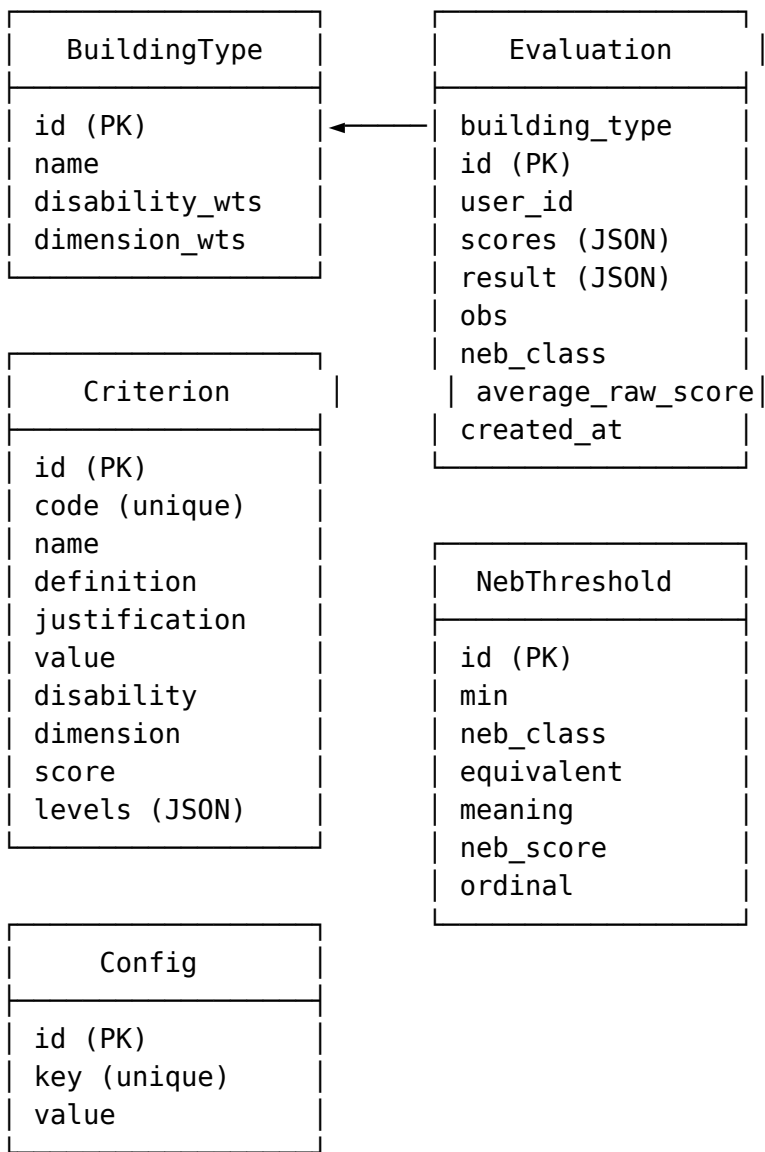
5. Config (config table)

Key-value store for system parameters.

Column	Type	Description
id	UUID (PK)	Auto-generated primary key
key	VARCHAR (unique)	Parameter key
value	TEXT	Parameter value

Known keys: MAX_OBS, DEFAULT_SCORE, STRENGTH_THRESHOLD, WEAKNESS_THRESHOLD, TOP_N_RESULTS.

Entity Relationship Diagram



Controller Layer

1. Evaluate Controller (evaluateController.ts)

Endpoint: POST /api/v1/evaluate

Purpose: The core evaluation endpoint. Accepts a building type and optional scores, runs the calculation engine, returns full evaluation results.

Request Body:


```

{
  "buildingType": "Commercial Buildings",
  "scores": {
    "EC1.1.1": 4,
    "EC1.1.2": 3
  }
}

```

Validation: - Body must be a JSON object - buildingType must be a string if provided (defaults to “Commercial Buildings”) - scores must be an object (Record<string, number>) if provided

Response: Returns { ok: true, result: {...} } with the full evaluation result.

2. Assessments Controller (assessmentsController.ts)

POST /api/v1/assessments — Create a new assessment.

- Accepts { buildingId, name }
- Looks up BuildingType by ID to get the name
- Runs calc.evaluate() with the building type name
- Creates an Evaluation record with scores, result, obs, neb_class, average_raw_score
- Returns 201 with { ok: true, data: { id, buildingId, name, results } }

GET /api/v1/assessments/:id — Retrieve an existing assessment.

- Looks up Evaluation by UUID
- Enriches with BuildingType ID
- Returns { ok: true, data: { id, buildingId, results } }
- Returns 404 if not found

3. Buildings Controller (buildingsController.ts)

GET /api/v1/buildings — List all building types with their assessment counts.

- Joins BuildingType and Evaluation data
- Groups evaluations by building_type name
- Returns array of { id, name, type, assessments: [...] }
- Assessment status label from ASSESSMENT_STATUS env var (default: "Completed")

GET /api/v1/buildings/:id — Get building detail with assessment history.

- Finds BuildingType by UUID
- Returns assessments sorted by created_at DESC
- Returns 404 if building not found

4. Data Controller (dataController.ts)

The largest controller — handles CRUD for reference data.

Function	Endpoint	Purpose
getCriteria	GET /criteria	List all criteria (sorted by code)
getCriterionByCode	GET /criteria/:code	Get single criterion by code
createCriterion	POST /criteria	Create new criterion with validation
getBuildingTypes	GET /building-types	List building types
getNebThresholds	GET /neb-thresholds	Get NEB classification bands
getConfig	GET /config	Get all system config entries
getDisabilityTypes	GET /disability-types	Get DT labels from JSON data file
getAssessmentDimensions	GET /assessment-dimensions	Get AD labels

Criterion Creation Validation: - code must match pattern /[^]EC\d+\.\d+\.\d+\$/ (e.g., EC1.1.3) - name and definition are required - levels must be an array of exactly 5 strings - disability must be between 1-5 - dimension must be between 1-5

5. Reports Controller (reportsController.ts)

GET /api/v1/reports — List evaluation reports.

- Returns evaluations sorted by created_at DESC
- Uses env vars for i18n:
 - REPORT_TITLE_PREFIX (default: "Accessibility Report")
 - REPORT_LOCALE (default: "en-US") for date formatting
- Title format: "{PREFIX} #{N} – {building_type} ({date}) – OBS: {obs}%"

6. Auth Controller (authController.ts)

POST /api/v1/login — Real token-based authentication via Node.js crypto.

Token Generation: - Uses crypto.randomBytes() with HMAC-SHA256 for access + refresh tokens - Token length: 48 hex characters - Configurable via JWT_SECRET (env, required), JWT_EXPIRES_IN (env,

default 3600), LOGIN_REDIRECT (env, default /dashboard) - Returns 500 with "Server configuration error: JWT_SECRET not set" if secret is missing

Validation: - email required, must contain @ - password required, minimum 6 characters

Response:

```
{
  "ok": true,
  "accessToken": "<48-char-hex-hmac>",
  "refreshToken": "<48-char-hex-hmac>",
  "expiresIn": 3600,
  "redirectTo": "/dashboard"
}
```

Service Layer

1. Calculation Service (calculationService.ts)

The core business logic engine. Exports a single function: evaluate(buildingType, scores?).

Algorithm

1. Load configuration (MAX_OBS, DEFAULT_SCORE, thresholds)
2. Load building type & its disability weights
3. Load all 63 criteria
4. Build criteria list with scores (user-provided or defaults)
5. Derive DT/AD ID sets dynamically from criteria data
6. For each criterion, compute $IS = \text{value} \times \text{score}$
7. Accumulate TIS per disability type: $TIS[i] = \sum IS_{ij}$
8. Accumulate CIS per dimension: $CIS[j] = \sum (DTWeight[i] \times IS_{ij})$
9. Compute $OBS = (\sum DTWeight[i] \times TIS[i]) / MAX_OBS \times 100$
10. Classify OBS into NEB band
11. Compute average scores per DT and per AD
12. Identify top strengths ($\text{score} \geq \text{STRENGTH_THRESHOLD}$)
13. Identify top weaknesses ($\text{score} \leq \text{WEAKNESS_THRESHOLD}$)
14. Compute score distribution (count per level 1-5)

Return Object

```
{
  obs: number; // Overall Building Score
  (0-100)
  nebClass: string; // "A+", "A", "B", "C",
  "D"
```

```

        nebEquivalent: string;           // Human-readable
    equivalent
        nebMeaning: string;             // What this class means
        nebScore: number;               // Numeric NEB score (1-
5)
        averageRawScore: number;       // Mean of all criterion
    scores
        avgRawScoreByDT: Record<number, number>; // Mean per DT
        avgRawScoreByAD: Record<number, number>; // Mean per AD
        strengths: Criterion[];         // Top N strongest
    criteria
        weaknesses: Criterion[];        // Top N weakest criteria
        scoreDistribution: Record<number, number>; // Counts per
    level
        tisByDT: Record<number, number>; // Type Impact Scores
        cisByAD: Record<number, number>; // Criterion Impact
    Scores
        criteria: Criterion[];          // All criteria with
    computed IS
    }

```

Key Helper Functions

`buildCriteriaFromList(list, scores?, defaultScore):` - Merges user-provided scores into criteria - Falls back to criterion's default score, then to `DEFAULT_SCORE` config value - Computes `is = value × score` for each criterion

`uniqueSortedIds(items, field):` - Dynamically derives the set of DT/AD IDs from actual criteria data - NOT hardcoded — adapts to whatever DT/AD values exist in the database

2. Metadata Service (`metadataService.ts`)

Provides cached access to reference data used by the calculation engine.

Functions

Function	Returns	Cache Strategy
<code>getNebThresholds()</code>	<code>NebThreshold[]</code>	In-memory cache (<code>cache.neb</code>)
<code>getConfigNumber(key, fallback)</code>	<code>number</code>	Per-key cache (<code>cache.config[key]</code>)
<code>getBuildingType(name)</code>	<code>BuildingType</code> <code>null</code>	No cache (direct query)
<code>getAllCriteria()</code>	<code>Criterion[]</code>	No cache (direct query)

Function	Returns	Cache Strategy
validateMetadata()	boolean	Verifies required data exists

validateMetadata()

Called at startup. Verifies: 1. MAX_OBS config key exists 2. At least 1 building type seeded 3. At least 1 criterion seeded 4. At least 1 NEB threshold seeded

Fails fast: If any check fails, the process exits with code 1 (process.exit(1)).

Cache Initialization

The service lazily initializes the DataSource. Repeated calls reuse the existing connection:

```
if (initialized || AppDataSource.isInitialized) return;
await AppDataSource.initialize();
initialized = true;
```

Calculation Engine

Complete Formula Reference

Notation: - DT(i) = Disability Type i (i = 1..5) - AD(j) = Assessment Dimension j (j = 1..5) - W(ij) = Cell weight for criterion at DT(i), AD(j) - S(ij) = User-assigned score (1-5) for criterion at DT(i), AD(j) - DW(i) = Disability weight for DT(i) from building type config

Individual Score:

$$IS_{ij} = W_{ij} \times S_{ij}$$

Type Impact Score (per disability type):

$$TIS_i = \sum_{j=1}^5 IS_{ij}$$

Criterion Impact Score (per assessment dimension):

$$CIS_j = \sum_{i=1}^5 DW_i \times IS_{ij}$$

Overall Building Score:

$$OBS = \frac{\sum_{i=1}^5 DW_i \times TIS_i}{\times 100} \{MAX_OBS\}$$

NEB Classification: - $OBS \geq 80 \rightarrow$ NEB Class A+ (Excellent, Score 5)
- $OBS \geq 65 \rightarrow$ NEB Class A (Very Good, Score 4) - $OBS \geq 50 \rightarrow$ NEB Class B (Good, Score 3) - $OBS \geq 35 \rightarrow$ NEB Class C (Fair, Score 2) - $OBS \geq 0 \rightarrow$ NEB Class D (Poor, Score 1)

Migrations

Three migration files in `src/migrations/`:

Migration	Timestamp	Purpose
1686300000000-InitSchema.ts	Initial	Creates all 5 tables
1686460000000-AddEvaluationFields.ts	+1	Adds obs, neb_class, average_raw_score to evaluations
1686550000000-AddNebAndConfig.ts	+2	Seeds NEB thresholds and config values

Running Migrations

```
cd backend
npx ts-node src/scripts/runMigrations.ts
```

Migrations are applied by TypeORM's migration runner in sequential order based on timestamps.

Database Seeding

Seed Script: `src/scripts/seed.ts`

Populates the database with initial reference data:

1. **Criteria** (63 records): Loaded from workbook mapping data
 - Each criterion has: code, name, definition, justification, value, disability, dimension, levels
2. **Building Types**: Default building categories with disability weights
3. **NEB Thresholds**: OBS classification bands
4. **Config**: System parameters (MAX_OBS, DEFAULT_SCORE, etc.)

Run Seed

```
cd backend
export
DATABASE_URL=mysql://myuser:mypassword@127.0.0.1:3306/mydb
npx ts-node src/scripts/seed.ts
```

Extraction Script:

src/scripts/extract_workbook_mappings.ts

Extracts EC-DT-AD mappings from the original Excel workbook and converts them to database-ready JSON for seeding.

Docker Configuration

docker-compose.yml

```
services:
  mariadb:
    image: mariadb:10.11
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: mydb
      MYSQL_USER: myuser
      MYSQL_PASSWORD: mypassword
    ports:
      - "3306:3306"
    volumes:
      - mariadb_data:/var/lib/mysql
      - ./docker-initdb.d:/docker-entrypoint-initdb.d
volumes:
  mariadb_data:
```

Init Script: docker-initdb.d/init.sql

```
CREATE DATABASE IF NOT EXISTS mydb;
CREATE USER IF NOT EXISTS 'myuser'@'%' IDENTIFIED BY
'mypassword';
GRANT ALL PRIVILEGES ON mydb.* TO 'myuser'@'%';
FLUSH PRIVILEGES;
```

Start/Stop

```
# Start
cd backend && docker compose up -d

# Stop
```

```
cd backend && docker compose down

# Stop & remove data
cd backend && docker compose down -v
```

Tests

Test Files

File	Purpose
calculation.test.ts	Unit tests for the calculation engine
metadata.test.ts	Unit tests for the metadata service
static.test.ts	Static data validation tests
workbook.mapping.test.ts	Workbook mapping integrity tests

Run Tests

```
cd backend
npm test                # Full suite with coverage
npm run test:automated  # CI-friendly output
```

Test Database

Tests use a separate database or in-memory configuration. Configure DATABASE_URL to point to a test database to avoid corrupting development data.

AASTool by Serg | Dev by Y